Brainstorming the architecture of asset management

1. Specifying system requirements

Mozaic system has aggregational and DAO use cases. Aggregational use cases are shown in the following figure:

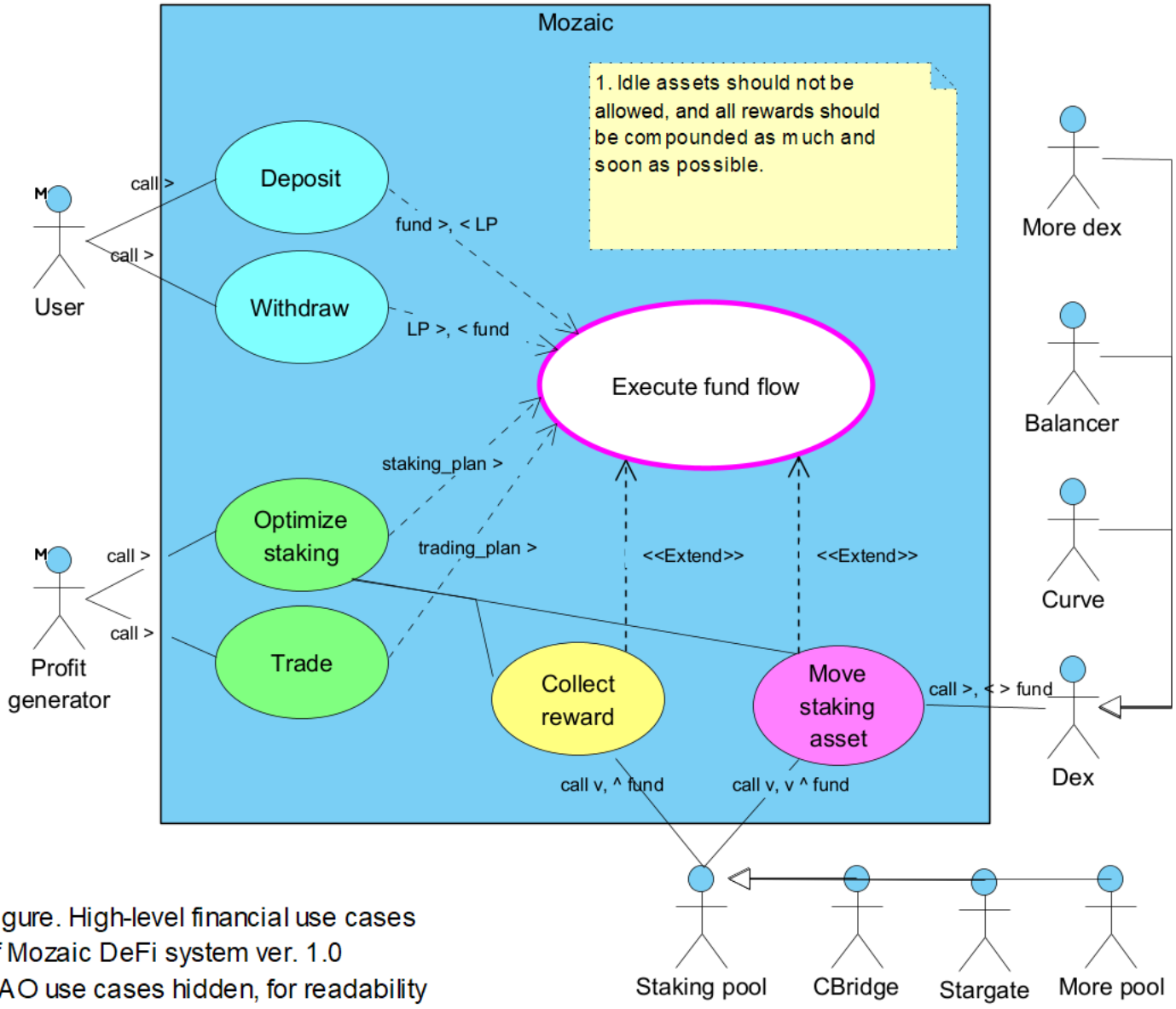

Figure. High-level financial use cases of Mozaic DeFi system ver. 1.0
DAO use cases hidden, for readability

The use cases and external actors are described below:

- **Compound**: This hidden use case compounds rewards. Idle assets should not ba allowed, and all rewards should be compounded as much and soon as possible.
- **Execute fund flow**: This use case checks, carry out, and keeps track of assets move. The assets managed by the system can only be moved by this use case transparently.

- **Deposit**: This use case deposit the user's assets in the system. **User** calls this use case in the hope that Mozaic system will **Optimize staking** of the deposited assets for them. This use case includes **Execute fund flow**.
- **Withdraw**: This use case withdraws from the user's rewards and, if needed, deposited assets. (The deposited assets are increased by perpetual compounding of rewards.) **User** calls this use case. This use case includes **Execute fund flow**.
- **Optimize staking**: This use case upgrades the staking of deposited assets to maximize rewards. **Profit generator**, a role of the system, calls this use case *either on a regular basis or at randome times*. This use case includes **Execute fund flow** and **Compound**. *By providing **Execute fund flow** with **staking_plan**, this use case effectively prevents it from being involved with finding optimal staking portfolio.*
- **Trade**: This use case swaps idle assets to get profit by using price changes. It calls **Dex**. *By providing **Execute fund flow** with **trading_plan**, this use case effectively prevents it from being involved with finding optimal trading orders.*
- **Collect reward**: This use case collects rewards from Staking pools. Use case Execute fund flow, when it is working under **Optimize staking**, is extended by this use case. This use case calls Staking pool.
- **Move staking asset**: This use case move assets to/between/from, **Staking pools**. **Execute fund flow**, when it is working under Optimize staking, is extended byt this use case. This use case calls Staking pool.
- **Dex**: This actor is a smart contract that swaps between assets. Examples are pairs on Curve and Balancer DeFies.
- **Staking pool**: This actor is a smart contract that allocates reward to assets that are staked in it. Examples are farming pools on CBridge and Stargate DeFies.

2. Identifying vaults through its surrounding modules

As the first implementation step, we identify the module that executes use case **Execute fund flow**, as it seems to act as the controller and be one of unique features of the system. The design decisions are as illustrated in the following figure:

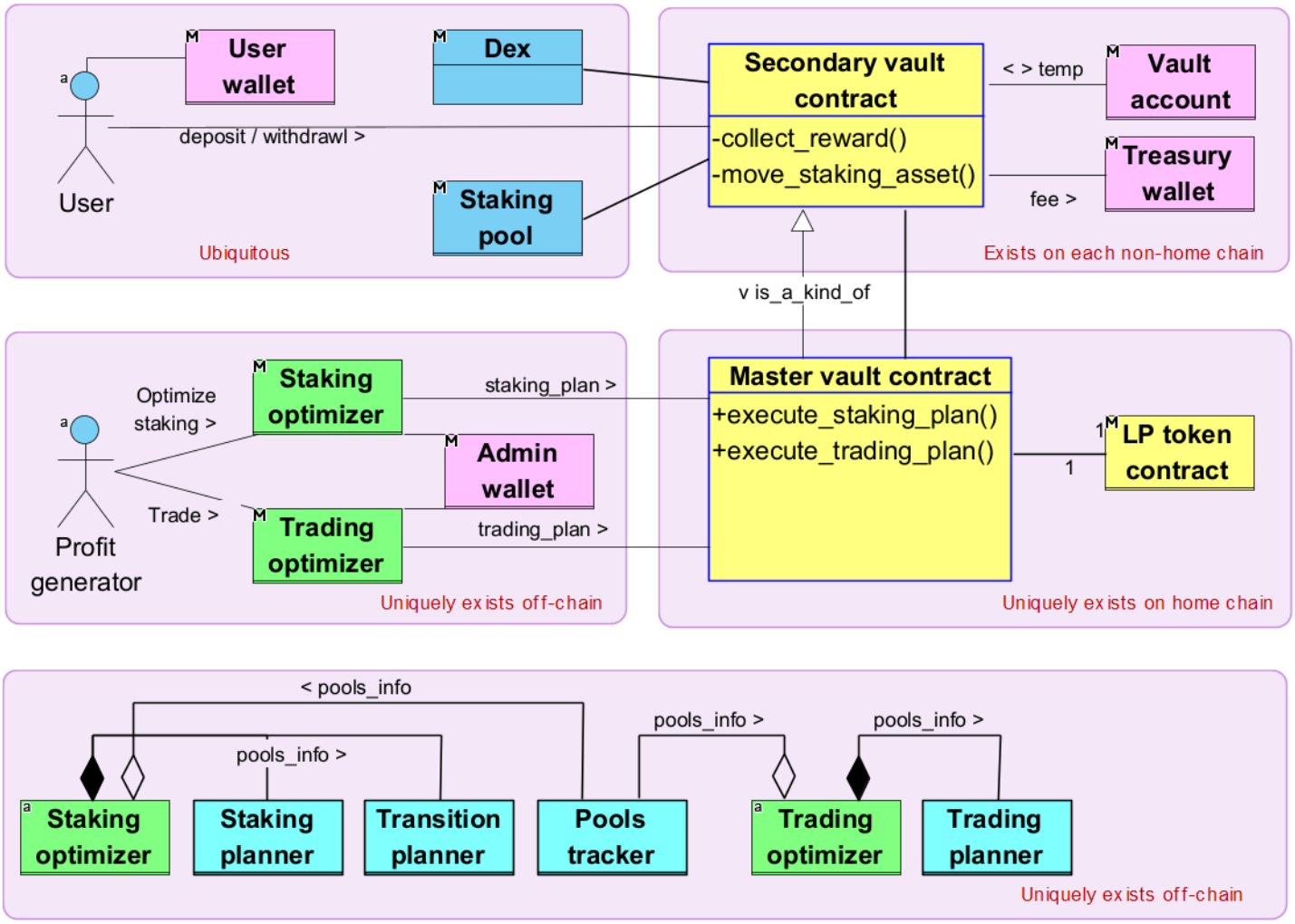

Figure. High-level functional modules of Mozaic DeFi ver. 1.0
Goal: Minimize the functionality of vaults.

Functional modules are described below:

- **Secondary valut contract**: This module is a smart contract and deployed on each chain except the home chain. It participates in the cooperation with its peer **Secondary vault contract**s to implement **Execute fund flow**, This module has at least the following private operations that work locally on its chain:
  - collect_reward(...)
  - move_staking_asset(...)

- **Master vault contract**: This module is a smart contract, has global operations, in addition to the local operations inherited from **Secondary vault contract**, and is deployed on one and only one of the chains, called **Home chain**, where it takes the role of **Secondary vault contract**, as well as the the unique role of the master vault operating **LP token contract**.
- **LP token contract**: This module manages the LP token balances of **User**s, which represents the their proportional share of the total assets of the system. Their share comprises of their deposited assets plus automatic compounding. *It is a deliberate design decision that the LP token only exists on **Home chain**.*
- **User wallet**: This is a blockchain wallet and identifies a **User**. **User**'s actions; like deposit, withdraw, and harvest; are authenticated/authorized with this wallet.
- **Vault account**: It is the blockchain account of, and controlled by, **Secondary vault contract** and used to store temporary assets, like funds pending staking.
- **Treasury wallet**: This is a blockchain wallet, and a place to store and retrieve system revenues, like fees. It will be better if it is not owned by a human, but be the account of a smart contract that only obeys vault contracts, for better decentralization. It is deployed on all chains.
- **Staking optimizer**: This is an off-chain module that can invoke **Master vault contract**. This module is globally unique, calculates optimal **staking_plan**s, and lets the master vault to execute the plans (in cooperation with secondary vaults). *It is an important design decision that the staking_plan is calculated off-chain, thus leading to transparency and security debates, for the sake of gas- and time- savings:*
  - Transparency debate: **User**s will not be able to track why the system chose particular **staking_plan**s technically.
  - Security debate: If the calculation of **staking_plan** is hacked or compromised, then the system will make a less-optimal staking manuever.
  - Justification: Only the second of the following concerns becomes less transparent, leading to both un-assured best profitability and assured huge gas- and time- savings.
    - how much of what assets from which pool to which pool, is the move about
    - whether all the asset moves are securely and/or reasonably/optimally chosen
    - whether all the asset moves are securely executed and logged
    - whether the move logs are readily available to check later
    - whether the execution of staking_plan is integrated
- **Trading optimizer**: This off-chain module is similar to **Staking optimizer**, except it relates to trading.
- **Adimin wallet**: This wallet is used to invoke **Master vault contract", in privilege, on behalf of the administrator.
- **Staking planner**: An integral component of **Staking optimizer**, this module predicts the next most profitable **staking_portfolio**, based on **pools_info** provided by **Pools tracker**. Running this module on-chain would enhance transparency, but would at the same time incur huge gas fee and effectively disable the system.
- **Transition planner**: An integral component of **Staking optimizer**, this module predicts the most efficient **staking_plan**, which is the best procedure of asset move that implements the transitioning to a given **staking_portfolio**, based on the current **pools_info**.
- **Trading optimizer**: This is similar to **Staking optimizer**, except that it relates trading.

- **Trading planner**: This is similar to **Staking planner**, except that it relates trading.
- **Pools tracker**: A shared module between **Staking optimizer** and **Trading optimizer**, this module retrieves and tracks all relevant information from chains, like Reward Release Speed, and total Staked LP of each pool. Running this module on-chain would enhance transparency, but would at the same time incur huge gas fee and effectively disable the system.

3. Exploring vaults

We have identified vaults through their surrounding modules interacting with them.
The external actors in the following use case diagram, together with their interactions with vaults are already described above. We can now explore the use cases of vault.

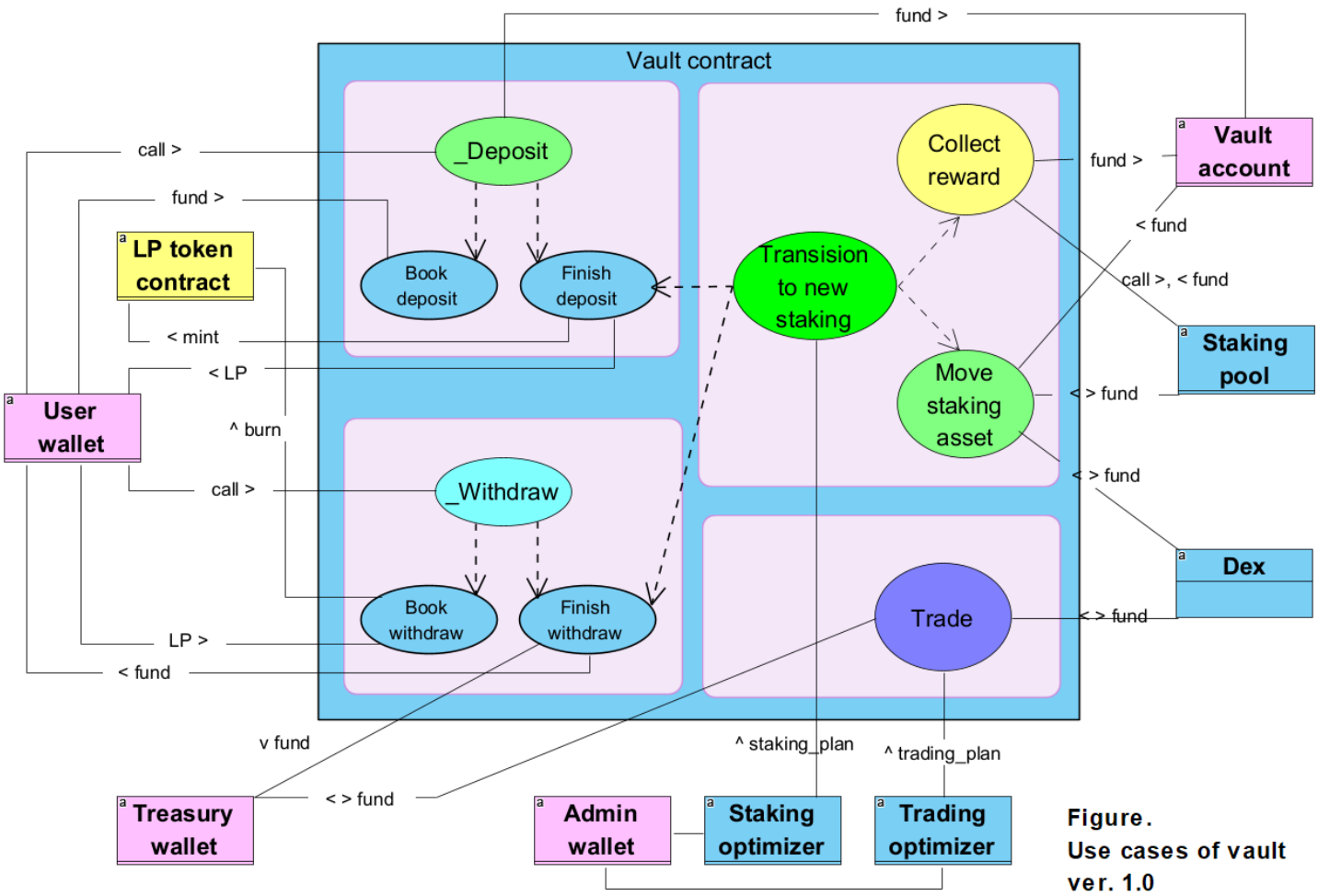

Figure.
Use cases of vault
ver. 1.0

- **_Deposit**: This is what happens at the level of vault contracts when **Deposit** use case is invoked at the system level. Invoked by **User wallet** with fund amount to deposit, this use case coordinates the following two use cases.
- **Book deposit**: This use case
  - collects the fund from **User wallet**,
  - books the deposit request with the system,
  - and pauses the session of "_Deposit".
- **Finish deposit**: This use case
  - resume the session of "_Deposit",
  - retrieves the booked deposit request,
  - mints LP tokens to cover the new fund,
  - returns the LP tokens to **User wallet**,

    - and helps **Transition to new staking** stake the fund.
- _**Withdraw**: This is what happens at the level of vault contracts when **Withdraw** use case is invoked at the system level. Invoked by **User wallet** wih LP tokens returned, this used case coordinates the following two use cases.
- **Book withdraw**: This use case
    - collects the returned LP tokens,
    - burns the collected LP tokens,
    - books the withdrawal request with the system,
    - and pauses the session of "_Deposit".
- **Finish deposit**: This use case
    - resume the session of "_Deposit",
    - retrieves the books withdrawal request,
    - subtract fund, as much as covered by the returned LP tokens, from the total system assets, and
    - returns the fund to **User wallet**
- **Transition to new staking**: This use case transitions to a new asset state by executing **staking_plan** provided by **Staking optimizer**. (This is the most challenging part of vault implementation.) It does *collectively*, by using **Move staking asset**,
    - **Collect reward**,
    - Collect staked assets, to cover the fund to **Finish withdraw**,
    - **Finish deposit**,
    - **Finish withdraw**,
    - execute remaining part of **staking_plan**
- **Trade**: This use case executes **trading_plan** provided by **Trading optimizer**.

4. Algorithm of Staking planner

1. **Definition**

- **Pools state**
  $PS = \{PS_i | i = 1..N\}$,
  where PS_i is the i-th pool state:
  $PS_i = (RewardRate_i, RewardToken_i, TotalStaking_i, StakingToken_i, MozaicStaking_i)$

- **Token vectors**
  - **User tokens vector**
    $UserTokens = \{UserToken_i | i = 1..M\}$
    Example: $UserTokens = \{USDT, USDC, ETH, BNB, BTC\}$
  - **Reward tokens vector**
    $RewardTokens = \{RewardToken_i | i = 1..N\}$
  - **Staking tokens vector**
    $StakingTokens = \{StakingToken_i | i = 1..N\}$
  - **Tokens vector**
    $Tokens = (UserTokens, RewardTokens, StakingTokens)$

- **Asset vectors**
  - **Vector of booked deposit requests**, at time index t
    $Deposits^t = \{D_i^t | D_i^t : Deposit\ amount, at\ time\ t,\ denominated\ by\ UserToken_i.i = 1..M\}$
    Example: $Deposits^t$ of {1, 2, 3} means {1 USDT, 2 USDC, 3 ETH}, assuming $UserTokens = \{USDT, USDC, ETH\}$
  - **Vector of booked withdrawals requests**, at time index t
    $Withdrawals^t = \{W_i^t | D_i^t :$
    $Withdrawal\ amount, at\ time\ t,\ denominated\ by\ UserToken_i.i = 1..M\}$
  - **Vector of collected rewards**, at time index t
    $Rewards^t = \{R_i^t | D_i^t :$
    $Reward\ amount, at\ time\ t,\ denominated\ by\ RewardToken_i.i = 1..M\}$
  - **Vector of staked assets**, at time index t
    $Stakes^t = \{S_i^t | D_i^t : Stake\ amount, at\ time\ t,\ denominated\ by\ StakingToken_i.i = 1..M\}$

- **Transformations**
  - $USDT^{Positive}$

$USDT^{Positive}$ transforms a Tokens-denominated asset vector to USDT-denominated vector, with market exchange rates.
Example: $USDT^{+}$ transforms (1, 2, 3) to (1, 2.02, 1300), assuming UserTokens = (USDT, USDC, ETH) and USDT/USDC = 1.01 and USDT/ETH = 1300.

- $USDT^{Negative}$
  $USDT^{Negative}$ transforms a USDT-denominated asset vector to Tokens-denominated vector, with market exchange rates.
  Example: $USDT^{-}$ transforms (1, 2.02, 1300) to (1, 2, 3), assuming UserTokens = (USDT, USDC, ETH) and USDT/USDC = 1.01 and USDT/ETH = 1300.
- FOP
  FOP, standing for Find Optimal Portfolio, finds the best USDT-denominated vector of staked assets, for a given total USDT amount.
  Example: FOP transforms (12345) to the best staking of 123 USDT over staking pools, and has the format of USDT-denominated $Stakes^t$, like (100, 20, 3) all in USDT, assuming we have total 3 pools.
- $ElementWiseSum$
  $ElementWiseSum$ sums up all elements of a vector.

- **Mozaic's asset state**, at time t
  $MS^t = (Tokens, Deposits^t, Withdrawals^t, Rewards^t, Stakes^t)$
  Or, simply, $MS^t = (T, D^t, W^t, R^t, S^t)$

2. **Formulae**

$$
\begin{array}{ccc}
MS^t=(T,\ D^t,\ W^t,\ R^t,\ S^t) & \xrightarrow{(transition)} & MS^{t+1}=(T,\ 0D,\ 0W,\ 0R,\ optimal\ S^{t+1}) \\
{\scriptstyle USDT^{Postitive}}\downarrow & & \uparrow{\scriptstyle USDT^{Negative}} \\
MS_U^t & \xrightarrow[(Implicit)]{} & MS_U^{t+1}=(T,\ 0D,\ 0W,\ 0R,\ FOP(Total\ USDT)) \\
{\scriptstyle ElementWiseSum}\downarrow & & \uparrow{\scriptstyle zeros,\ FOP} \\
Total\ USDT & \xrightarrow{Identity\ transformation} & Total\ USDT
\end{array}
$$

, where 0D, 0D, and 0R are a vector of zero values in their respective vector lengths.

The whole algorithm is a chain of transformations:
$optimial\ S^{t+1} = USDT^{Negative} * FOP * ElementWiseSum * USDT^{Positive}(T,\ D^t,\ W^t,\ R^t,\ S^t)$

- $USDT^{Positive}$ and $USDT^{Negative}$ are obvious, except that we may need systematical methods to find best Dexes and swap paths.
- FOP is solved analytically, demonstrating about 9% of competitive edge over the public.
- ElementWiseSum is trivial.
- $D^t and W^t$ can be retrieved from the booked requests of deposits and withdrawals.
- S^t is found when we "Collect reward" pending rewards.

5. Algorithm of Transition planner